Connecting Earth Engine to myGeoID

How the Google Earth Engine connection was fixed, 2026-08-16/17, and what to do if it ever breaks again. Every number here was measured, not recalled.

The short version

Nine faults, in two groups. **Five were in the code** and are fixed. **Four were configuration in the Google console**, which no amount of code could resolve and which only the account owner can change.

The single most useful thing learnt: **the error message names which of the two groups you are in**, and they are easy to tell apart once you know the list.

Part 1 — the Google console

Three separate settings must all be right. They live on different pages, and getting one wrong produces an error that sounds like one of the others.

1. The OAuth client and its JavaScript origins

Google Auth Platform → Clients → your Web client → Authorised JavaScript origins.

Add the **exact** origin the page is served from:

```
http://localhost:8100
http://127.0.0.1:8100
```

No trailing slash, no path, and `http` not `https` — Google matches the string verbatim. `http://localhost:8100/` is a different origin and will be refused.

This is not the redirect-URI box. Authorised redirect URIs sit directly

below and are used by a different flow; entries put there do nothing for a browser app and were a genuine source of confusion here.

Origin changes take **5 minutes to a few hours** to reach all of Google's front-ends. During that window the authorisation endpoint can accept an origin that the sign-in popup still refuses — a real state that looks exactly like a mistake.

2. The audience — test users

Google Auth Platform → Audience → Test users.

While the consent screen is in **Testing**, only listed addresses may sign in.

Everyone else gets `403 access_denied`, including the project owner.

Add the signing-in address:

```
geoid.initiative@gmail.com
```

Effective immediately, with no propagation delay.

Do not press "Publish app" instead. The Earth Engine scope is classed as sensitive, so publishing starts Google's verification review — weeks of process — for something one person needs. Testing mode allows up to 100 test users and is the correct route for an internal or in-development app.

3. The Cloud project

The Earth Engine API must be enabled on the project, and the project id goes in the app's Settings:

```
geoid-504623
```

Part 2 — tokens, and why there is no API key

Earth Engine does not support API keys. The API accepts only:

method	credential	needs a server?
user OAuth	the signed-in user	no
service account	a key file	**yes** — a key cannot be served to a page

A page served to a browser cannot hold a secret, so a browser app must use OAuth. That is why the origins allowlist matters so much: it is the *only* thing protecting a public Client ID.

Signing in is once per browser, not once per session

After the first consent Google remembers both the session and the grant. Later token requests return without user action, and the token refreshes itself. You should see the popup once.

The temporary access token

Settings carries an **Access token (temporary)** field. It exists only as a bridge for testing while OAuth is broken:

```
gcloud auth application-default print-access-token
```

It lasts about an hour and **takes precedence over sign-in**, so leaving one there silently prevents OAuth from ever being exercised. Clear it once sign-in works.

Never paste a token into a chat, an issue or a commit. To revoke one that has been exposed:

```
gcloud auth application-default revoke
```

Part 3 — the code faults, and what each looked like

All fixed; recorded because each was invisible until a real call was made.

1. **No quota-project header.** Every call returned `403` — "requires a quota

project, which is not set by default". Fixed by sending `x-goog-user-project`.

2. **An invented request shape.** The client sent a readable object (`{collection, band, reducer, window, region}`). Earth Engine takes a **serialised expression graph** (`{values, result}`, each node a `functionInvocationValue`) and answered `Unknown name "collection" at 'expression': Cannot find field`.

3. **A function that does not exist.** `Collection.filterDate` is not an EE algorithm. The API's own list — `GET /v1/projects/{p}/algorithms`, 985 of them — gives `Collection.filter` + `Filter.dateRangeContains` + `DateRange`. **Ask the API rather than guessing; it answered in one step what guessing had not in several.**

4. **A band that is not there.** `total_precipitation_surface` is not on these images. `precipitation_rate` is — and it is better physics: a rate is instantaneous, so millimetres per hour is `rate × 3600`, and a time step needs no difference against the run start. The cumulative-within-a-run subtraction the pipeline was designed around is not needed at all.

5. **Two modules claimed one global.** `gee.js` assigned `window.GeoIDEarthEngine` and loaded *after* `gee-live.js`, silently replacing it. From the page that is indistinguishable from a module that failed to load. Both now merge with `Object.assign`.

Part 4 — diagnosing it again

Two committed tests. Between them they separate configuration from code, which is the distinction that cost the most time here.

Is the console right?

```
python3 GeoID_GIS/tests/gee-oauth.py <client-id>
```

Asks Google's authorisation endpoint what the client accepts. No credentials and no sign-in — it validates the client *before* authenticating anyone.

Read the last line, `code postmessage`, which tests the **JavaScript origins** list:

result	meaning
`ACCEPTED`	the origin is registered
`invalid_client`	the client has **no** origins registered
`redirect_uri_mismatch` on that line	the origin is not among those registered

The ten `code` lines above it test *redirect URIs*, which this app does not use; mismatches there are expected and correct.

Is the API path right?

```
python3 GeoID_GIS/tests/gee-live.py
```

Sends the same request the browser builds, from Python, over a token from your gcloud login. Four stages, each failing differently:

1. auth and quota project
2. the band the client asks for exists
3. a real GFS grid over the study area
4. **consecutive steps differ**

Stage 4 is the important one. A series that repeats a single frame looks alive and is exactly the "static map" symptom this project chased for days.

Measured when it first passed:

```
2048 cells   max 3.119 mm/h   mean 0.254 mm/h   raining 1109/2048
00h 0.237 mm/h   06h 0.012 mm/h   12h 0.050 mm/h
```

Part 5 — reading the error

The failure names the fault. This table is the session's hard-won summary.

error	what it means	where to fix
`400 invalid_request`	the origin is one Google refuses over http — e.g. `0.0.0.0`	serve at `localhost`
`401 invalid_client` / "no registered origin"	the client has **no** JavaScript origins at all	console → Clients
`400 origin_mismatch`	origins exist; the page's address is not among them	compare `location.origin`
`403 access_denied`	consent screen is in Testing and you are not a test user	console → Audience
`403` quota project	the request lacks `x-goog-user-project`	code (fixed)
`400 Unknown name …`	the expression is not a serialised graph	code (fixed)
`400 Band pattern … did not match`	wrong band name	code (fixed)

Lessons worth keeping

- **Measure the environment; do not infer it.** Several hours went into reasoning about caches, iframes and module wiring when one probe would have said the origins list was empty.
- **`location.origin` is the whole question** whenever the error mentions an origin. It must match the console string character for character.
- **A test written from the same assumption as the code cannot catch that assumption.** Nine unit tests passed while pinning a request shape Earth Engine rejects on sight — they were checking an invention against itself.
- **Ask the API what it offers.** Its algorithm list and its band list each resolved in one step what guessing had not.
- **Never truncate an error body.** A 400-character cut hid the correct band

name past the end of a list and manufactured a false failure.